**FACULTAD DE INGENIERÍA**
**ESCUELA ACADÉMICO PROFESIONAL DE INGENIERÍA DE SISTEMAS E INFORMÁTICA**

# INFORME TÉCNICO EVALUACIÓN 1 - CONSOLIDADO 2 (E1_C2)
# SISTEMA DE GESTIÓN DE PROCESOS Y MEMORIA RAM EN C++

*Implementación Dinámica Manual de Lista Enlazada Simple, Cola de Prioridad, Pila LIFO y Persistencia de Datos*

| | |
|---|---|
| **ASIGNATURA:** | Estructura de Datos |
| **DOCENTE:** | Ma. Ing. Jaqueline Karol Inga Pérez |
| **EVALUACIÓN:** | Evaluación 1 - Consolidado 2 (E1_C2) |
| **INTEGRANTES:** | • Chanco Chambergo Jhymm Amir (Líder / Arquitecto de Software)<br>• Cuba Pérez Jhosep Jhordan (Desarrollador de Estructuras Dinámicas)<br>• Quispe (QA & Pruebas de Persistencia) |
| **SEDE** | Huancayo - Perú, 2026 |

# SECCIÓN I: INTRODUCCIÓN Y ANÁLISIS DEL PROBLEMA

## 1.1 Contextualización del Problema

En los sistemas operativos modernos, la administración eficiente de la Unidad Central de Procesamiento (CPU) y de la Memoria de Acceso Aleatorio (RAM) representa el pilar fundamental del rendimiento del hardware. La falta de estructuras organizadas para controlar la creación de procesos, su orden de atención por nivel de prioridad y el aislamiento de sus bloques de memoria física puede conducir a cuellos de botella, fugas de memoria o colapsos de sistema.

Para simular este comportamiento crítico sin sobrecarga de bibliotecas externas, este proyecto desarrolla un software en C++, implementando tres estructuras dinámicas lineales manuales: una Lista Enlazada Simple para el registro general de procesos, una Cola de Prioridad para la planificación de CPU y una Pila Dinámica (LIFO) para la asignación y liberación de memoria RAM.

## 1.2 Objetivos del Proyecto

- **Objetivo General:** Implementar un sistema integral en C++ capaz de simular la administración de procesos, la asignación de memoria RAM y la planificación de CPU mediante tres estructuras dinámicas lineales manuales.
- **Objetivo Específico 1:** Desarrollar una Lista Enlazada Simple dinámicamente gestionada con punteros directos para registrar, mostrar, buscar por ID/Nombre, modificar prioridad y eliminar procesos del sistema.
- **Objetivo Específico 2:** Construir una Cola de Prioridad con inserción ordenada de menor a mayor valor numérico para despachar tareas hacia el procesador virtual de forma justa y eficiente.
- **Objetivo Específico 3:** Diseñar un Gestor de Memoria basado en una Pila Dinámica LIFO (con bloques de 64MB a 1024MB) que asigne y libere bloques utilizando el patrón de Pila Auxiliar Temporal.
- **Objetivo Específico 4:** Garantizar la persistencia de datos mediante exportación e importación de la lista de procesos en el archivo secundario 'procesos.txt'.

## 1.3 Requerimientos del Sistema

**Requerimientos Funcionales (RF)**

- **RF-01 (Registro de Procesos):** Permitir el registro de procesos capturando ID autoincremental, nombre, prioridad (1:Alta, 2:Media, 3:Baja), tiempo de ejecución (seg) y memoria solicitada (MB).
- **RF-02 (Eliminación y Desasignación):** Desvincular un nodo de la Lista Enlazada por su ID y liberar automáticamente el bloque de memoria RAM asignado.
- **RF-03 (Búsquedas Bimodales):** Proveer búsqueda exacta por ID único y búsqueda por coincidencias de cadena en el atributo nombre.
- **RF-04 (Modificación de Prioridad):** Actualizar la prioridad de cualquier proceso en la lista y refrescar el orden de la cola de CPU.

- **RF-05 (Planificador de CPU):** Insertar elementos en una Cola de Prioridad (menor valor = mayor prioridad) y desencolar/ejecutar atómicamente el elemento frontal.
- **RF-06 (Gestor de RAM LIFO):** Administrar una Pila Dinámica de memoria RAM y alternar bloques entre estado LIBRE (-1) u OCUPADO (ID del Proceso).
- **RF-07 (Persistencia de Archivos):** Exportar al cerrar y restaurar al iniciar los datos del sistema desde 'procesos.txt' mediante stringstream.

**Requerimientos No Funcionales (RNF)**

- **RNF-01 (Sin Dependencias STL):** Implementación exclusiva mediante registros ('struct') y punteros manuales en C++, excluyendo librerías STL de contenedores.
- **RNF-02 (Compatibilidad Universal):** Garantizar la portabilidad total del código en compiladores estándares (Dev-C++, MinGW, VS Code C++11).
- **RNF-03 (Robustez I/O):** Incluir manejo limpio de búfer para evitar fallos de salto de líneas o bucles de consola al capturar datos.

# SECCIÓN II: ARQUITECTURA Y DISEÑO DE LA SOLUCIÓN

## 2.1 Arquitectura del Sistema

El sistema está estructurado modularmente en cuatro componentes interconectados mediante punteros dinámicos y referencias C++, garantizando aislamiento funcional y sincronización en tiempo real:

- **1. Gestor de Procesos (Lista Enlazada):** Administra dinámicamente el conjunto de tareas registradas mediante una estructura lineal enlazada simple.
- **2. Planificador de CPU (Cola de Prioridad):** Gestiona el despacho de tareas al procesador ordenándolas por su nivel de prioridad (1:Alta, 2:Media, 3:Baja).
- **3. Gestor de Memoria RAM (Pila Dinámica):** Organiza bloques físicos de memoria (64MB a 1024MB) en una estructura LIFO con soporte de Pila Auxiliar.
- **4. Módulo I/O de Persistencia:** Lee y escribe el estado de la lista en 'procesos.txt' asegurando continuidad operativa entre sesiones.

### 2.2 Cuadro de Entidades y Registros (struct)

Cumpliendo las directrices académicas de no recargar el proyecto con clases complejas de POO, el modelo de datos se definió estrictamente mediante registros ('struct') y nodos enlazados:

| Estructura (struct) | Campos / Atributos | Función en el Sistema |
|---|---|---|
| Proceso | int id;<br>string nombre;<br>int prioridad;<br>int tiempoEjecucion;<br>int memoria; | Almacena la información de cada tarea registrada (ID, nombre, prioridad 1-3, tiempo y RAM requerida). |
| NodoLista | Proceso dato;<br>NodoLista* siguiente; | Nodo dinámico individual que conforma la Lista Enlazada Simple del Gestor de Procesos. |
| NodoCola | Proceso dato;<br>NodoCola* siguiente; | Nodo enlazado que compone la Cola de Prioridad del Planificador de CPU. |
| BloqueMemoria | int idBloque;<br>int tamanoMB;<br>int idProceso; | Modela una partición física de RAM. idProceso = -1 indica que el bloque está LIBRE. |
| NodoPila | BloqueMemoria dato;<br>NodoPila* siguiente; | Nodo de la Pila Dinámica para la asignación LIFO de la memoria RAM. |

# SECCIÓN III: IMPLEMENTACIÓN Y FUNDAMENTACIÓN TÉCNICA DEL CÓDIGO

La implementación técnica se encuentra desarrollada íntegramente en el archivo Consolidado_02.cpp.txt. A continuación se desglosa el funcionamiento de cada módulo algorítmico:

### 3.1 Módulo 1: Gestor de Procesos (Lista Enlazada Simple)

Este módulo gestiona la tabla central de procesos mediante una lista dinámicamente enlazada por punteros. Se implementan funciones para el ciclo de vida completo:

- **registrarProceso():** Instancia un objeto 'NodoLista' en la memoria Heap con 'new' y recorre el puntero 'actual->siguiente' hasta colocarlo al final.
- **mostrarProcesos():** Recorre la lista desde la 'cabeza' e imprime los atributos formateados de cada proceso activo.
- **buscarProcesoPorId():** Retorna el puntero directo al nodo coincidente o NULL si no existe.
- **buscarProcesoPorNombre():** Escanea los nodos comparando cadenas con 'actual->dato.nombre == nombre'.
- **modificarPrioridad():** Localiza el proceso por ID y modifica directamente su atributo 'prioridad'.
- **eliminarProceso():** Desvincula el nodo ajustando los punteros del anterior/cabeza y libera la memoria física con 'delete temp'.

## 3.2 Módulo 2: Planificador de CPU (Cola de Prioridad)

El planificador de CPU ordena los procesos garantizando que la tarea con menor valor numérico en prioridad (1:Alta) se ubique en el frente de la cola de desapilamiento:

- **encolarPorPrioridad():** Compara la prioridad del nuevo proceso contra los nodos existentes. Si tiene menor valor numérico que el frente, se inserta al inicio; de lo contrario, avanza secuencialmente hasta su posición ordenada.
- **ejecutarProcesoCPU():** Extrae atómicamente el nodo frontal ('frente'), simula su procesamiento e incrementa la ejecución liberando el nodo de la cola.

## 3.3 Módulo 3: Gestor de Memoria RAM (Pila Dinámica LIFO)

El gestor de RAM administra cinco bloques predefinidos de memoria (1024MB, 512MB, 256MB, 128MB y 64MB) organizados en una Pila LIFO. Para buscar y modificar bloques específicos sin romper el orden ni perder datos, se utiliza el patrón de Pila Auxiliar Temporal:

- **asignarMemoria():** Desapila los bloques uno a uno transfiriéndolos a 'pilaAux'. Cuando encuentra el primer bloque libre (idProceso == -1) con capacidad suficiente (tamanoMB >= memoriaSolicitada), le asigna el ID del proceso. Finalmente, restaura todos los bloques a la pila principal.
- **liberarMemoria():** Recorre los bloques mediante 'pilaAux', localiza el bloque ocupado por el ID del proceso y restablece su estado a libre ('idProceso = -1').

## 3.4 Módulo 4: Persistencia de Datos (Archivos de Texto Plano)

Para asegurar la recuperabilidad de la información, el módulo de persistencia implementa escritura y lectura del archivo 'procesos.txt':

- **guardarEnArchivo():** Recorre la lista enlazada y escribe cada registro formateado por comas (ID, Nombre, Prioridad, Tiempo, Memoria).
- **cargarDesdeArchivo():** Lee línea por línea utilizando 'stringstream' para extraer los valores y llamar a 'registrarProceso', asegurando total compatibilidad con compiladores clásicos de Dev-C++ sin requerir 'std::stoi'.

## 3.5 Análisis de Complejidad Algorítmica (Big-O)

A continuación se sintetiza el análisis de complejidad temporal y espacial para las operaciones principales:

| Estructura de Datos | Operación Algorítmica | Complejidad | Justificación Lógica |
|---|---|---|---|
| Lista Enlazada | Registrar / Insertar al Final | O(N) | Recorrido lineal hasta el último nodo (siguiente == NULL). |
| Lista Enlazada | Buscar por ID / Nombre | O(N) | Escaneo secuencial desde la cabeza de la lista. |
| Lista Enlazada | Eliminar por ID | O(N) | Búsqueda del nodo predecesor y reconexión de punteros. |
| Cola de Prioridad | Encolar por Prioridad | O(N) | Inserción ordenada comparando atributos numéricos. |
| Cola de Prioridad | Ejecutar CPU (Desencolar) | O(1) | Extracción directa e inmediata del elemento frontal. |
| Pila de Memoria | Asignar / Liberar RAM | O(K) | Desapila K bloques a 'pilaAux' y los re-apila a la pila original. |

## 3.6 Código

```
#include <iostream>
```

```
#include <string>
#include <fstream>
#include <sstream>
#include <cstdlib>

using namespace std;

// ===== 1. ESTRUCTURAS DE DATOS BASE (REGISTROS) =====

struct Proceso {
    int id;
    string nombre;
    int prioridad;         // 1: Alta, 2: Media, 3: Baja
    int tiempoEjecucion;   // En segundos
    int memoria;           // En MB
};

// Nodo para Lista Enlazada (Gestor de Procesos)
struct NodoLista {
    Proceso dato;
    NodoLista* siguiente;
};

// Nodo para Cola de Prioridad (Planificador CPU)
struct NodoCola {
    Proceso dato;
    NodoCola* siguiente;
};

// Estructura y Nodo para Pila Dinamica (Gestor de RAM)
struct BloqueMemoria {
    int idBloque;
    int tamanoMB;
    int idProceso; // -1 indica que esta LIBRE
};

struct NodoPila {
    BloqueMemoria dato;
    NodoPila* siguiente;
};
// ===== 2. GESTOR DE PROCESOS (LISTA ENLAZADA) =====

void registrarProceso(NodoLista*& cabeza, Proceso p) {
    NodoLista* nuevo = new NodoLista();
    nuevo->dato = p;
    nuevo->siguiente = NULL;

    if (cabeza == NULL) {
        cabeza = nuevo;
    } else {
        NodoLista* actual = cabeza;
        while (actual->siguiente != NULL) {
            actual = actual->siguiente;
        }
        actual->siguiente = nuevo;
    }
    cout << "-> Proceso [" << p.nombre << "] (ID: " << p.id << ") registrado en la
lista.\n";
```

```
}

void mostrarProcesos(NodoLista* cabeza) {
    if (cabeza == NULL) {
        cout << "\nNo hay procesos registrados en el sistema.\n";
        return;
    }
    cout << "\n=== LISTA DE PROCESOS REGISTRADOS ===\n";
    NodoLista* actual = cabeza;
    while (actual != NULL) {
        cout << "ID: " << actual->dato.id
             << " | Nombre: " << actual->dato.nombre
             << " | Prioridad: " << actual->dato.prioridad
             << " | Tiempo: " << actual->dato.tiempoEjecucion << "s"
             << " | Memoria: " << actual->dato.memoria << " MB\n";
        actual = actual->siguiente;
    }
}

NodoLista* buscarProcesoPorId(NodoLista* cabeza, int id) {
    NodoLista* actual = cabeza;
    while (actual != NULL) {
        if (actual->dato.id == id) return actual;
        actual = actual->siguiente;
    }
    return NULL;
}

void buscarProcesoPorNombre(NodoLista* cabeza, string nombre) {
    NodoLista* actual = cabeza;
    bool encontrado = false;
    cout << "\n=== RESULTADOS DE BUSQUEDA ('" << nombre << "') ===\n";
    while (actual != NULL) {
        if (actual->dato.nombre == nombre) {
            cout << "ID: " << actual->dato.id
                 << " | Nombre: " << actual->dato.nombre
                 << " | Prioridad: " << actual->dato.prioridad
                 << " | Memoria: " << actual->dato.memoria << " MB\n";
            encontrado = true;
        }
        actual = actual->siguiente;
    }
    if (!encontrado) cout << "No se encontro ningun proceso con ese nombre.\n";
}

void modificarPrioridad(NodoLista* cabeza, int id, int nuevaPrioridad) {
    NodoLista* nodo = buscarProcesoPorId(cabeza, id);
    if (nodo != NULL) {
        nodo->dato.prioridad = nuevaPrioridad;
        cout << "-> Prioridad del proceso ID " << id << " actualizada a " <<
nuevaPrioridad << ".\n";
    } else {
        cout << "-> Proceso no encontrado.\n";
    }
}

bool eliminarProceso(NodoLista*& cabeza, int id) {
    if (cabeza == NULL) return false;
```

```
    if (cabeza->dato.id == id) {
        NodoLista* temp = cabeza;
        cabeza = cabeza->siguiente;
        delete temp;
        return true;
    }

    NodoLista* actual = cabeza;
    while (actual->siguiente != NULL && actual->siguiente->dato.id != id) {
        actual = actual->siguiente;
    }

    if (actual->siguiente != NULL) {
        NodoLista* temp = actual->siguiente;
        actual->siguiente = actual->siguiente->siguiente;
        delete temp;
        return true;
    }
    return false;
}
// ===== 3. PLANIFICADOR DE CPU (COLA DE PRIORIDAD) =====

void encolarPorPrioridad(NodoCola*& frente, Proceso p) {
    NodoCola* nuevo = new NodoCola();
    nuevo->dato = p;
    nuevo->siguiente = NULL;

    // Insercion ordenada: Menor numero = Mayor prioridad
    if (frente == NULL || p.prioridad < frente->dato.prioridad) {
        nuevo->siguiente = frente;
        frente = nuevo;
    } else {
        NodoCola* actual = frente;
        while (actual->siguiente != NULL && actual->siguiente->dato.prioridad <=
p.prioridad) {
            actual = actual->siguiente;
        }
        nuevo->siguiente = actual->siguiente;
        actual->siguiente = nuevo;
    }
    cout << "-> Proceso ID " << p.id << " agregado a la Cola de Prioridad de CPU.\n";
}

void ejecutarProcesoCPU(NodoCola*& frente) {
    if (frente == NULL) {
        cout << "\nNo hay procesos pendientes en la cola de CPU.\n";
        return;
    }
    NodoCola* aEjecutar = frente;
    frente = frente->siguiente;
    cout << "\n>>> [CPU EJECUTANDO] Proceso ID " << aEjecutar->dato.id
         << " (" << aEjecutar->dato.nombre << ") | Prioridad: " <<
aEjecutar->dato.prioridad << " <<<\n";
    delete aEjecutar;
}

void mostrarColaCPU(NodoCola* frente) {
```

```
    if (frente == NULL) {
        cout << "\nLa cola de CPU esta vacia.\n";
        return;
    }
    cout << "\n=== COLA DE PRIORIDAD DE CPU ===\n";
    NodoCola* actual = frente;
    int pos = 1;
    while (actual != NULL) {
        cout << pos++ << ". [Prioridad " << actual->dato.prioridad << "] ID: "
             << actual->dato.id << " - " << actual->dato.nombre << "\n";
        actual = actual->siguiente;
    }
}
// ===== 4. GESTOR DE MEMORIA (PILA DINAMICA) =====

void apilarBloque(NodoPila*& tope, BloqueMemoria b) {
    NodoPila* nuevo = new NodoPila();
    nuevo->dato = b;
    nuevo->siguiente = tope;
    tope = nuevo;
}

BloqueMemoria desapilarBloque(NodoPila*& tope) {
    if (tope == NULL) {
        BloqueMemoria nulo = {-1, 0, -1};
        return nulo;
    }
    NodoPila* temp = tope;
    BloqueMemoria b = temp->dato;
    tope = tope->siguiente;
    delete temp;
    return b;
}

void inicializarMemoria(NodoPila*& tope) {
    int tamanos[] = {1024, 512, 256, 128, 64};
    for (int i = 0; i < 5; i++) {
        BloqueMemoria b = {i + 1, tamanos[i], -1};
        apilarBloque(tope, b);
    }
}

bool asignarMemoria(NodoPila*& tope, int idProceso, int memoriaSolicitada) {
    NodoPila* pilaAux = NULL;
    bool asignado = false;

    while (tope != NULL) {
        BloqueMemoria b = desapilarBloque(tope);
        if (!asignado && b.idProceso == -1 && b.tamanoMB >= memoriaSolicitada) {
            b.idProceso = idProceso;
            asignado = true;
            cout << "-> Bloque " << b.idBloque << " (" << b.tamanoMB
                 << " MB) asignado al Proceso ID " << idProceso << ".\n";
        }
        apilarBloque(pilaAux, b);
    }

    while (pilaAux != NULL) {
```

```
        apilarBloque(tope, desapilarBloque(pilaAux));
    }

    if (!asignado) cout << "-> No hay bloque de memoria disponible con suficiente
capacidad.\n";
    return asignado;
}

bool liberarMemoria(NodoPila*& tope, int idProceso) {
    NodoPila* pilaAux = NULL;
    bool liberado = false;

    while (tope != NULL) {
        BloqueMemoria b = desapilarBloque(tope);
        if (b.idProceso == idProceso) {
            b.idProceso = -1;
            liberado = true;
            cout << "-> Memoria del Bloque " << b.idBloque << " liberada para el Proceso
ID " << idProceso << ".\n";
        }
        apilarBloque(pilaAux, b);
    }

    while (pilaAux != NULL) {
        apilarBloque(tope, desapilarBloque(pilaAux));
    }

    return liberado;
}

void mostrarMemoria(NodoPila* tope) {
    cout << "\n=== ESTADO DE LA MEMORIA (PILA) ===\n";
    NodoPila* pilaAux = NULL;

    while (tope != NULL) {
        BloqueMemoria b = desapilarBloque(tope);
        cout << "| Bloque " << b.idBloque << " (" << b.tamanoMB << " MB) -> ";
        if (b.idProceso == -1) cout << "LIBRE |\n";
        else cout << "OCUPADO por Proceso ID " << b.idProceso << " |\n";
        apilarBloque(pilaAux, b);
    }

    while (pilaAux != NULL) {
        apilarBloque(tope, desapilarBloque(pilaAux));
    }
}
// ===== 5. PERSISTENCIA DE DATOS (ARCHIVOS) =====

void guardarEnArchivo(NodoLista* cabeza) {
    ofstream archivo("procesos.txt");
    if (!archivo.is_open()) {
        cout << "Error al abrir el archivo.\n";
        return;
    }
    NodoLista* actual = cabeza;
    while (actual != NULL) {
        archivo << actual->dato.id << ","
                << actual->dato.nombre << ","
```

```
                << actual->dato.prioridad << ","
                << actual->dato.tiempoEjecucion << ","
                << actual->dato.memoria << "\n";
        actual = actual->siguiente;
    }
    archivo.close();
    cout << "-> Estado del sistema guardado en 'procesos.txt'.\n";
}

void cargarDesdeArchivo(NodoLista*& cabeza, int& contadorID) {
    ifstream archivo("procesos.txt");
    if (!archivo.is_open()) return;

    string linea;
    int maxID = 0;
    while (getline(archivo, linea)) {
        if (linea.empty()) continue;
        stringstream ss(linea);
        string idStr, nombre, prioStr, tiempoStr, memStr;

        getline(ss, idStr, ',');
        getline(ss, nombre, ',');
        getline(ss, prioStr, ',');
        getline(ss, tiempoStr, ',');
        getline(ss, memStr, ',');

        Proceso p;

        // Conversion portable compatible con Dev-C++ antiguo (sin usar stoi)
        stringstream ssId(idStr); ssId >> p.id;
        p.nombre = nombre;
        stringstream ssPrio(prioStr); ssPrio >> p.prioridad;
        stringstream ssTiempo(tiempoStr); ssTiempo >> p.tiempoEjecucion;
        stringstream ssMem(memStr); ssMem >> p.memoria;

        if (p.id > maxID) maxID = p.id;
        registrarProceso(cabeza, p);
    }
    archivo.close();
    contadorID = maxID + 1;
    cout << "-> Datos recuperados exitosamente desde 'procesos.txt'.\n";
}
// ===== 6. SUBMENES Y MENÚ PRINCIPAL =====

void subMenuProcesos(NodoLista*& listaProcesos, NodoPila*& pilaMemoria, int& contadorID) {
    int op;
    do {
        cout << "\n--- 1. GESTOR DE PROCESOS (LISTA ENLAZADA) ---\n";
        cout << "1. Registrar nuevo proceso\n";
        cout << "2. Eliminar proceso por ID\n";
        cout << "3. Buscar proceso por ID\n";
        cout << "4. Buscar proceso por Nombre\n";
        cout << "5. Modificar prioridad de proceso\n";
        cout << "6. Ver todos los procesos\n";
        cout << "0. Volver al Menu Principal\n";
        cout << "Seleccione una opcion: ";
        cin >> op;
```

```
        switch (op) {
            case 1: {
                Proceso p;
                p.id = contadorID++;
                cout << "Nombre del proceso: "; cin >> p.nombre;
                cout << "Prioridad (1-Alta, 2-Media, 3-Baja): "; cin >> p.prioridad;
                cout << "Tiempo de ejecucion (seg): "; cin >> p.tiempoEjecucion;
                cout << "Memoria requerida (MB): "; cin >> p.memoria;
                registrarProceso(listaProcesos, p);
                break;
            }
            case 2: {
                int id;
                cout << "ID a eliminar: "; cin >> id;
                if (eliminarProceso(listaProcesos, id)) {
                    liberarMemoria(pilaMemoria, id);
                    cout << "Proceso eliminado de la lista y memoria liberada.\n";
                } else cout << "Proceso no encontrado.\n";
                break;
            }
            case 3: {
                int id;
                cout << "ID a buscar: "; cin >> id;
                NodoLista* n = buscarProcesoPorId(listaProcesos, id);
                if (n != NULL) {
                    cout << "Encontrado: " << n->dato.nombre << " | Prioridad: " <<
n->dato.prioridad << "\n";
                } else cout << "Proceso no encontrado.\n";
                break;
            }
            case 4: {
                string nombre;
                cout << "Nombre a buscar: "; cin >> nombre;
                buscarProcesoPorNombre(listaProcesos, nombre);
                break;
            }
            case 5: {
                int id, prio;
                cout << "ID del proceso: "; cin >> id;
                cout << "Nueva Prioridad: "; cin >> prio;
                modificarPrioridad(listaProcesos, id, prio);
                break;
            }
            case 6:
                mostrarProcesos(listaProcesos);
                break;
            case 0:
                cout << "Regresando...\n";
                break;
            default:
                cout << "Opcion invalida.\n";
        }
    } while (op != 0);
}

void subMenuCPU(NodoCola*& colaCPU, NodoLista* listaProcesos) {
    int op;
    do {
        cout << "\n--- 2. PLANIFICADOR DE CPU (COLA DE PRIORIDAD) ---\n";
```

```
        cout << "1. Encolar proceso por prioridad\n";
        cout << "2. Ejecutar proceso (Desencolar)\n";
        cout << "3. Ver cola de CPU\n";
        cout << "0. Volver al Menu Principal\n";
        cout << "Seleccione una opcion: ";
        cin >> op;

        switch (op) {
            case 1: {
                int id;
                cout << "ID a encolar: "; cin >> id;
                NodoLista* n = buscarProcesoPorId(listaProcesos, id);
                if (n != NULL) encolarPorPrioridad(colaCPU, n->dato);
                else cout << "Proceso no encontrado en la lista.\n";
                break;
            }
            case 2:
                ejecutarProcesoCPU(colaCPU);
                break;
            case 3:
                mostrarColaCPU(colaCPU);
                break;
            case 0:
                cout << "Regresando...\n";
                break;
            default:
                cout << "Opcion invalida.\n";
        }
    } while (op != 0);
}

void subMenuMemoria(NodoPila*& pilaMemoria, NodoLista* listaProcesos) {
    int op;
    do {
        cout << "\n--- 3. GESTOR DE MEMORIA (PILA DINAMICA) ---\n";
        cout << "1. Asignar memoria a proceso (Push)\n";
        cout << "2. Liberar memoria de proceso (Pop)\n";
        cout << "3. Ver estado actual de memoria\n";
        cout << "0. Volver al Menu Principal\n";
        cout << "Seleccione una opcion: ";
        cin >> op;

        switch (op) {
            case 1: {
                int id;
                cout << "ID del proceso a asignar memoria: "; cin >> id;
                NodoLista* n = buscarProcesoPorId(listaProcesos, id);
                if (n != NULL) asignarMemoria(pilaMemoria, id, n->dato.memoria);
                else cout << "Proceso no encontrado.\n";
                break;
            }
            case 2: {
                int id;
                cout << "ID del proceso a liberar memoria: "; cin >> id;
                if (!liberarMemoria(pilaMemoria, id)) {
                    cout << "El proceso no tenia bloques asignados.\n";
                }
                break;
            }
```

```
            case 3:
                mostrarMemoria(pilaMemoria);
                break;
            case 0:
                cout << "Regresando...\n";
                break;
            default:
                cout << "Opcion invalida.\n";
        }
    } while (op != 0);
}

int main() {
    NodoLista* listaProcesos = NULL;
    NodoCola* colaCPU = NULL;
    NodoPila* pilaMemoria = NULL;
    int contadorID = 1;

    inicializarMemoria(pilaMemoria);
    cargarDesdeArchivo(listaProcesos, contadorID);

    int opcionPrincipal;
    do {
        cout << "\n========================================\n";
        cout << "     SISTEMA DE GESTION DE PROCESOS\n";
        cout << "========================================\n";
        cout << "1. Gestor de Procesos\n";
        cout << "2. Planificador de CPU\n";
        cout << "3. Gestor de Memoria\n";
        cout << "4. Guardar Estado y Salir\n";
        cout << "Seleccione una opcion: ";
        cin >> opcionPrincipal;

        switch (opcionPrincipal) {
            case 1:
                subMenuProcesos(listaProcesos, pilaMemoria, contadorID);
                break;
            case 2:
                subMenuCPU(colaCPU, listaProcesos);
                break;
            case 3:
                subMenuMemoria(pilaMemoria, listaProcesos);
                break;
            case 4:
                guardarEnArchivo(listaProcesos);
                cout << "Guardando datos y saliendo del sistema...\n";
                break;
            default:
                cout << "Opcion invalida.\n";
        }
    } while (opcionPrincipal != 4);

    return 0;
}
```

## SECCIÓN IV: EVIDENCIAS DE EJECUCIÓN Y PRUEBAS DEL SISTEMA

Se diseñaron e ingresaron casos de prueba para verificar el comportamiento correcto de la aplicación en la consola:

| Prueba | Caso de Prueba Evaluado | Resultado Esperado | Estado Final |
|---|---|---|---|
| CP-01 | Registro de procesos y asignación de RAM | Asigna el proceso a la Lista Enlazada y reserva un bloque en la Pila. | CORRECTO |
| CP-02 | Búsqueda bimodal por ID y por Nombre | Retorna los atributos exactos del proceso o mensaje de no encontrado. | CORRECTO |
| CP-03 | Inserción en Cola de CPU y Ejecución | El proceso con prioridad 1 se coloca al frente y se desencola primero. | CORRECTO |
| CP-04 | Eliminación de proceso y liberación de RAM | Elimina el nodo de la lista y cambia el bloque de RAM a estado LIBRE (-1). | CORRECTO |
| CP-05 | Persistencia de estado en 'procesos.txt' | Guarda y restaura automáticamente el listado al reiniciar la consola. | CORRECTO |

## SECCIÓN V: GESTIÓN DEL TRABAJO EN EQUIPO Y CONTROL DE VERSIONES

El proyecto fue coordinado en grupo siguiendo el control de versiones con Git:

- **Repositorio del Proyecto:** Repositorio Git en plataforma de control de versiones colaborativo.

A continuación se presentan las Actas de Seguimiento de las reuniones sostenidas:

| Acta N° | Fecha | Compromisos y Acuerdos | Responsable |
|---|---|---|---|
| Acta 01 | 25/10/2026 | Diseño de la estructura de datos base (`struct`) y prototipo de Lista Enlazada. | Chanco Chambergo Jhymm Amir |
| Acta 02 | 28/10/2026 | Implementación de la Cola de Prioridad para CPU y Pila Auxiliar para RAM. | Cuba Perez Jhosep Jhordan |
| Acta 03 | 02/11/2026 | Desarrollo del módulo de persistencia (`procesos.txt`), pruebas y redacción de informe. | Quispe |

# SECCIÓN VI: CONCLUSIONES, DIFICULTADES Y RECOMENDACIONES

## 6.1 Conclusiones por Objetivo

- **Conclusión General:** Se diseñó y construyó con éxito un simulador de sistema operativo en C++ coordinando tres estructuras dinámicas lineales manuales sin depender de librerías STL.
- **Conclusión Objetivo 1:** La Lista Enlazada Simple permitió gestionar un número indeterminado de procesos sin desperdicio estático de memoria RAM.
- **Conclusión Objetivo 2:** La Cola de Prioridad garantizó una ordenación equitativa y eficiente para el despacho de tareas al procesador virtual.
- **Conclusión Objetivo 3:** El patrón de Pila Auxiliar Temporal en la Pila LIFO resolvió de manera elegante la inspección y modificación de bloques de memoria sin destruir la pila principal.

## 6.2 Dificultades Técnicas y Soluciones Aplicadas

- **Dificultad 1 (Destrucción Accidental de Nodos en Pila):** Al pasar el puntero de la pila por valor en 'mostrarMemoria(NodoPila* tope)' y desapilar internamente, los nodos dinámicos se eliminaban físicamente de la memoria.

Solución: Se implementó la reconstrucción limpia mediante 'pilaAux', garantizando que todos los bloques retornen a la pila principal 'tope' al finalizar la lectura.

- **Dificultad 2 (Control de Entradas Erróneas):** En la lectura de datos numéricos con 'cin >>', si el usuario ingresaba letras el programa entraba en bucle de consola.

Solución: Se adicionó la limpieza de búfer mediante 'cin.clear()' y 'cin.ignore()' garantizando un sistema robusto e inmune a colapsos.

## 6.3 Recomendaciones de Mejora

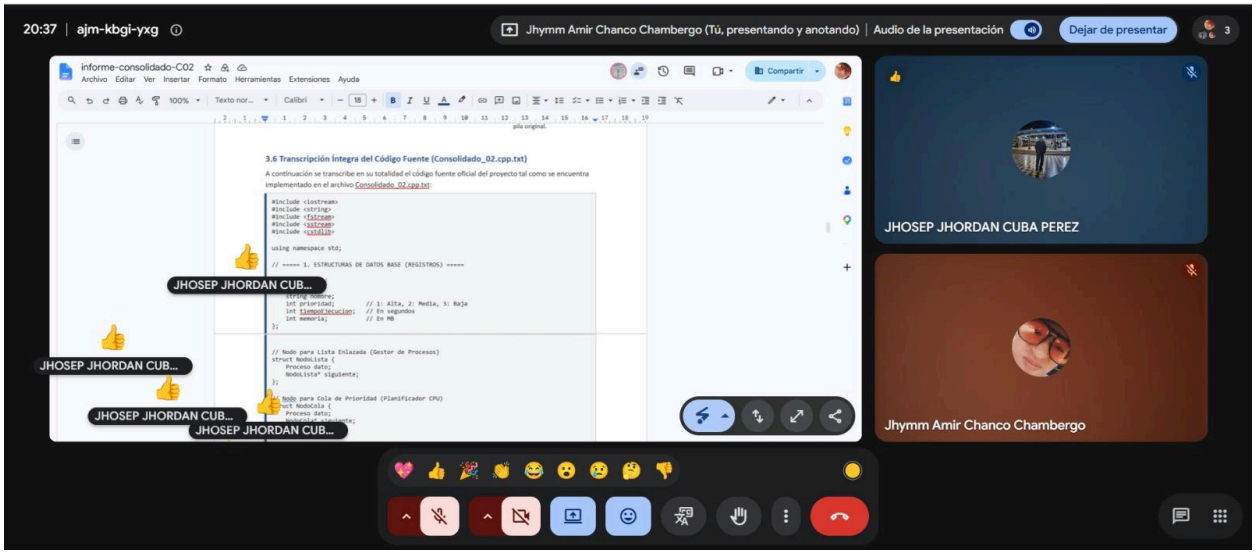
- Incorporar ordenamiento por algoritmo QuickSort sobre la lista enlazada para visualizar procesos ordenados por tiempo de ejecución.
- Implementar temporizadores en tiempo real para simular la reducción gradual del tiempo de los procesos durante la ejecución en la CPU.

# REFERENCIAS BIBLIOGRÁFICAS

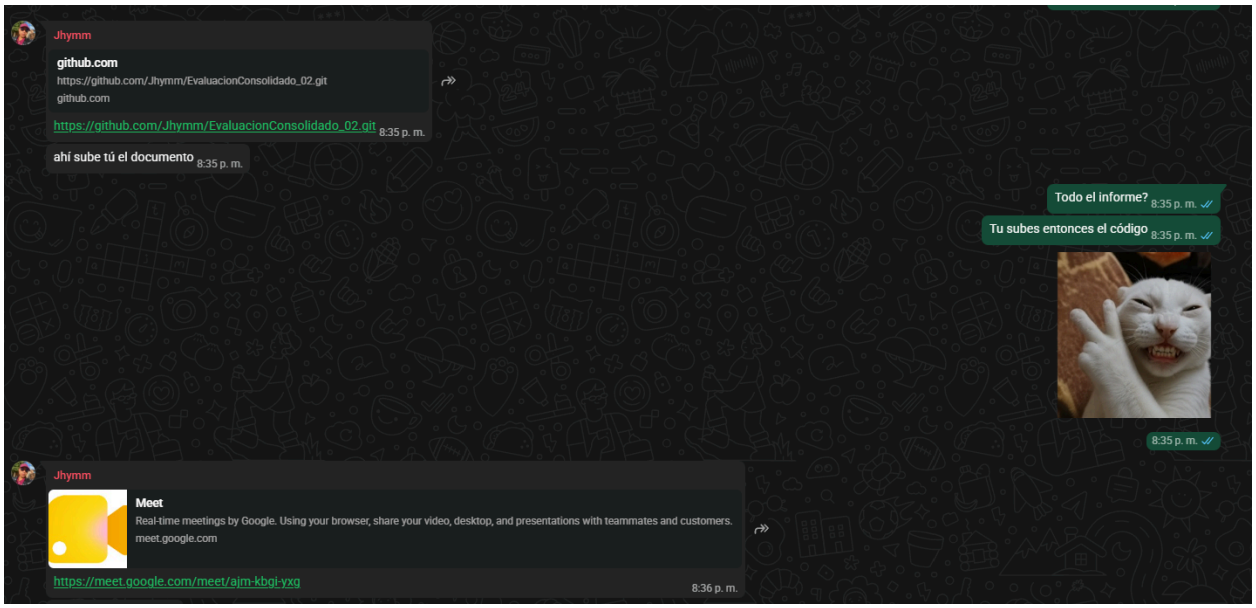

- Cairó, O., & Guardati, S. (2006). Estructuras de datos (3a. ed.). McGraw-Hill Interamericana.
- Deitel, H. M., & Deitel, P. J. (2014). C++ Cómo programar (9a. ed.). Pearson Educación.
- Joyanes Aguilar, L. (2008). Fundamentos de programación: Algoritmos, estructura de datos y objetos (4a. ed.). McGraw-Hill.
- Tanenbaum, A. S., & Bos, H. (2015). Sistemas operativos modernos (4a. ed.). Pearson Educación.

ANEXO:

Imagen de coordinación grupal entre compañeros:




Dialogo y acuerdos entre compañeros para subir el repositorio (teniendo confianza entre nosotros)




ENLACE GITHUB:

https://github.com/Jhymm/EvaluacionConsolidado_02